

## 1. Rilevamento e Mitigazione di Attacchi DoS in Reti SDN

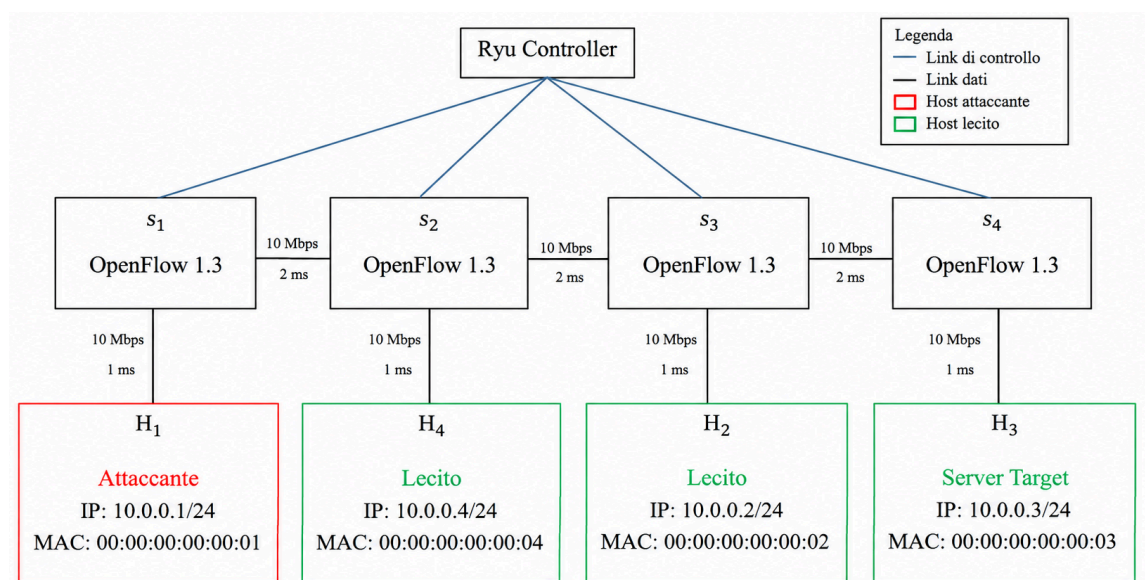
Nelle reti tradizionali, un dispositivo intermedio (come uno switch o un router) decide autonomamente come inoltrare i pacchetti analizzando le proprie tabelle interne. Se un attaccante genera un volume anomalo di traffico, bloccare l'intera porta di ingresso dello switch rischia di disconnettere anche tutti gli utenti legittimi attestati su quel medesimo nodo (problema noto come over-blocking).

In questo progetto realizzeremo un'architettura SDN (Software-Defined Networking) in cui il piano di controllo (Control Plane) è separato e centralizzato rispetto al piano dati (Data Plane). L'obiettivo è monitorare periodicamente il volume di traffico transitante su ciascuna porta della rete e, al rilevamento di un'inondazione di pacchetti non autorizzata, installare in tempo reale una regola di scarto (DROP) mirata esclusivamente all'indirizzo dell'attaccante, preservando integralmente il traffico degli utenti leciti.

### 1.1 Componenti del Sistema

L'ambiente di sperimentazione si articola su tre elementi fondamentali:

- Emulatore di Rete (Mininet): Ricrea una topologia virtuale composta da quattro switch OpenFlow e quattro host terminali, nello specifico: l'attaccante  $H_1$ , gli utenti legittimi  $H_2$  e  $H_4$ , e il terzo nodo  $H_3$  che funge da server di destinazione.
- Controller SDN (Ryu): Rappresenta l'intelligenza centrale della rete. Comunica con gli switch mediante il protocollo OpenFlow 1.3, gestendo la tabella di instradamento e interrogando regolarmente i dispositivi per calcolare il volume e la frequenza del traffico (Mbps e PPS), isolando in tempo reale le eventuali minacce.
- Generatore di Traffico (iperf3): Strumento software utilizzato per simulare sia una sessione dati standard affidabile (TCP), sia un attacco di saturazione volumetrica mediante flood ad alta intensità (UDP).



- Il controller viene eseguito. In particolare viene avviato il framework SDN Ryu (ryu-manager) utilizzando l'interprete Python 3.9 per eseguire l'applicazione personalizzata `sdn_defense_controller.py`.

```
giulia@RETI:~$ cd ~/sdn_project
python3.9 /usr/local/bin/ryu-manager sdn_defense_controller.py
```

- Viene emulata un'infrastruttura di rete virtuale tramite Mininet, collegandola a un controller SDN esterno.

```
giulia@RETI:~$ cd ~/sdn_project
sudo mn -c
sudo python3 custom_topology.py
[sudo] password for giulia:
```

```
def run():
    topo = ProjectTopo()
    # Connessione al Ryu Controller remoto in ascolto sulla porta standard 6653 o 6633
    net = Mininet(topo=topo, link=TCLink, switch=OVSSwitch, controller=None)
    net.addController('c0', controller=RemoteController, ip='127.0.0.1', port=6633)

    net.start()
    info('*** Rete avviata. Controller atteso su 127.0.0.1:6633\n')
    CLI(net)
    net.stop()
```

- Si testa la connettività di rete tramite il comando pingall.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> xterm h1 h4 h3
```

- Si utilizza iperf3 per generare e misurare il traffico di rete.

- $H_3$  viene avviato come server.

```
root@RETI:/home/giulia/sdn_project# iperf3 -s
```

- Il nodo  $H_4$  è configurato come client per generare un flusso di traffico verso il server  $H_3$  (10.0.0.3). Il test ha una durata complessiva di 60 secondi (-t 60) e prevede la stampa a video delle metriche prestazionali a intervalli di campionamento periodici di 1 secondo (-i 1).

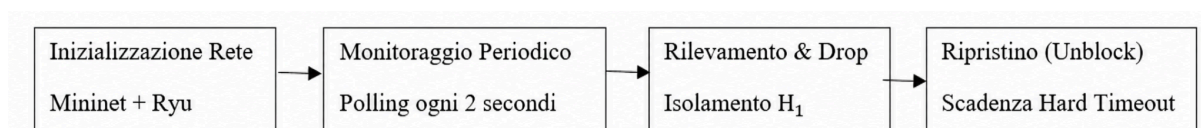
```
root@RETI:/home/giulia/sdn_project# iperf3 -c 10.0.0.3 -t 60 -i 1
```

- Il nodo  $H_1$  è configurato come client per simulare un attacco verso il server  $H_3$  (10.0.0.3). Il test inietta traffico non affidabile basato su protocollo UDP (-u) forzando un bitrate di trasmissione costante pari a 10 Mbps (-b 10M) per una durata complessiva di 30 secondi (-t 30). UDP, a differenza di TCP, è privo di algoritmi di regolazione della finestra, quindi  $H_1$  continua a trasmettere alla "cieca".

```
root@RETI:/home/giulia/sdn_project# iperf3 -c 10.0.0.3 -u -b 10M -t 30
```

## 1.2 Fasi operative

L'esecuzione del progetto segue una sequenza controllata in quattro passi:



1. All'avvio della simulazione, gli switch OpenFlow non possiedono ancora regole per gestire i nuovi flussi dati. Inizialmente, tramite la funzione `switch_features_handler`, il controller installa su ciascuno switch una regola predefinita di **Table-Miss** a priorità 0 (`priority=0`), la quale ordina di inoltrare al controller (OFPP\_CONTROLLER) qualunque pacchetto sconosciuto.

```
def switch_features_handler(datapath: Any):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    self.datapaths[datapath.id] = datapath

    match = parser.OFPMatch()
    actions = [parser.OFPACTIONOutput(ofproto.OFPP_CONTROLLER, ofproto.OFPCML_NO_BUFFER)]
    self.add_flow(datapath, priority=0, match=match, actions=actions)
    self.logger.info("Switch registrato: DPID=%016x", datapath.id)
```

2. Non appena gli host iniziano a trasmettere, lo switch intercetta i primi pacchetti e, non trovando corrispondenze nella propria Flow Table, li incapsula e li trasmette al controller Ryu. Il controller fa poi richiesta, ogni 2 secondi, delle statistiche agli switch che glielo forniscono.
3. Il controller estrae tali statistiche, le confronta con la misurazione precedente e, se necessario, avvia la procedura di mitigazione e blocco del flusso.

```
if prev:
    delta_bytes = total_bytes - prev['bytes']
    delta_packets = total_packets - prev['packets']
    delta_time = current_time - prev['time']

    if delta_time > 0:
        throughput_mbps = (delta_bytes * 8.0) / (delta_time * 1000000.0)
        pps = delta_packets / delta_time

        # **AGGIUNTO: Controllo combinato Mbps e PPS per distinguere l'attacco**
        if throughput_mbps > self.THRESHOLD_MBPS and pps > self.THRESHOLD_PPS:
            self.logger.warning("[ALLARME TRAFFICO] Switch=%016x Porta=%d Throughput=%.2f Mbps | PPS=%.1f",
                                dpid, port_no, throughput_mbps, pps)
```

```
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.30 Mbps | PPS=779.9
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.38 Mbps | PPS=788.0
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.40 Mbps | PPS=788.2
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.73 Mbps | PPS=816.1
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.59 Mbps | PPS=804.3
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.46 Mbps | PPS=793.7
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.50 Mbps | PPS=796.8
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=9.57 Mbps | PPS=802.7
! [ALLARME TRAFFICO] Switch=0000000000000001 Porta=1 Throughput=8.95 Mbps | PPS=751.2
```

4. La mitigazione attiva è il meccanismo di difesa reattivo con cui il controller SDN interviene direttamente sulla rete per neutralizzare una sorgente di traffico anomala nel momento in cui questa supera contemporaneamente le due soglie consentite. A tale scopo, il controller deve conoscere la posizione esatta della sorgente, causa di quel traffico anomalo.

```
for ip, location in self.ip_to_location.items():
    if location == (dpid, port_no):
        attacker_ip = ip
        break
```

```
>>> [MITIGAZIONE ATTIVA] Flow DROP installato su DPID=0000000000000001 per IP=10.0.0.1 (Durata: 20s)
>>> [MITIGAZIONE ATTIVA] Flow DROP installato su DPID=0000000000000003 per IP=10.0.0.1 (Durata: 20s)
>>> [MITIGAZIONE ATTIVA] Flow DROP installato su DPID=0000000000000002 per IP=10.0.0.1 (Durata: 20s)
>>> [MITIGAZIONE ATTIVA] Flow DROP installato su DPID=0000000000000004 per IP=10.0.0.1 (Durata: 20s)
```

Una volta identificato l'indirizzo malevolo, il controller avvia l'isolamento della sorgente invocando la funzione `add_drop_flow` che installa una regola con priorità elevata (`priority=100`). Poiché la lista delle azioni associate è vuota (`actions=[]`), lo switch esegue uno scarto immediato (*DROP*) di tutti i pacchetti emessi dall'attaccante. Per evitare

l'esclusione permanente dell'host, la regola di scarto viene configurata con durata massima di 20 secondi (`hard_timeout=20`).

```
def add_drop_flow(self, datapath, src_ip, hard_timeout):
    parser = datapath.ofproto_parser
    match = parser.OFPMatch(eth_type=ether_types.ETH_TYPE_IP, ipv4_src=src_ip)
    self.add_flow(datapath, priority=100, match=match, actions=[], hard_timeout=hard_timeout)
    self.logger.info(">>> [MITIGAZIONE ATTIVA] Flow DROP installato su DPID=%016x per IP=%s (Durata: %ds)",
                     datapath.id, src_ip, hard_timeout)
```

5. Allo scadere dei 20 secondi, viene invocato il metodo `_unblock_ip` per rimuovere l'indirizzo dalla lista dei bloccati, consentendo la progressiva ripresa delle normali comunicazioni di rete.

```
def _unblock_ip(self, ip):
    if ip in self.blocked_ips:
        self.blocked_ips.remove(ip)
        self.logger.info("<<< [UNBLOCK] Periodo di mitigazione scaduto per IP=%s. Ripristino traffico.", ip)
    <<< [UNBLOCK] Periodo di mitigazione scaduto per IP=10.0.0.1. Ripristino traffico.
```

Analizziamo il traffico generato da  $H_1$  e  $H_4$ , e il traffico ricevuto da  $H_3$ .

Poiché  $H_1$  utilizza il protocollo UDP che non implementa alcun meccanismo di controllo della congestione né attende riscontri (ACK), il trasmettitore inietta un flusso costante a 10 Mbps (> 4 Mbps) per tutti i 30 secondi, generando circa 863 (>650) datagrammi al secondo per un volume totale di 35.8 MB (0/25898 pacchetti persi lato sender). Il controller rileva il superamento di entrambe le soglie e installa la regola di DROP, limitando significativamente il traffico UDP ricevuto dal server H3.

```
root@RETI:/home/giulia/sdn_project# iperf3 -c 10.0.0.3 -u -b 10M -t 30
Connecting to host 10.0.0.3, port 5201
```

| [ ID] | Interval        | Transfer    | Bitrate        | Total Datagrams |
|-------|-----------------|-------------|----------------|-----------------|
| [ 7]  | 0.00-1.00 sec   | 1.19 MBytes | 9.96 Mbits/sec | 863             |
| [ 7]  | 1.00-2.00 sec   | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 2.00-3.00 sec   | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 3.00-4.00 sec   | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 4.00-5.00 sec   | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 5.00-6.00 sec   | 1.19 MBytes | 9.98 Mbits/sec | 862             |
| [ 7]  | 6.00-7.00 sec   | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 7.00-8.00 sec   | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 8.00-9.00 sec   | 1.19 MBytes | 9.99 Mbits/sec | 863             |
| [ 7]  | 9.00-10.00 sec  | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 10.00-11.00 sec | 1.19 MBytes | 9.99 Mbits/sec | 863             |
| [ 7]  | 11.00-12.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 12.00-13.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 13.00-14.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 14.00-15.00 sec | 1.19 MBytes | 9.99 Mbits/sec | 863             |
| [ 7]  | 15.00-16.00 sec | 1.19 MBytes | 9.99 Mbits/sec | 862             |
| [ 7]  | 16.00-17.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 865             |
| [ 7]  | 17.00-18.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 18.00-19.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 19.00-20.00 sec | 1.19 MBytes | 9.99 Mbits/sec | 863             |
| [ 7]  | 20.00-21.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 862             |
| [ 7]  | 21.00-22.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 22.00-23.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 23.00-24.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 24.00-25.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |
| [ 7]  | 25.00-26.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 864             |
| [ 7]  | 26.00-27.00 sec | 1.19 MBytes | 9.97 Mbits/sec | 860             |
| [ 7]  | 27.00-28.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 865             |
| [ 7]  | 28.00-29.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 865             |
| [ 7]  | 29.00-30.00 sec | 1.19 MBytes | 10.0 Mbits/sec | 863             |

```

[ ID] Interval      Transfer      Bitrate      Jitter      Lost/Total Datag
rams
[ 7]  0.00-30.00 sec  35.8 MBytes  10.0 Mbits/sec  0.000 ms  0/25898 (0%) se
nder
```

```
root@RETI:/home/giulia/sdn_project# iperf3 -s
Server listening on 5201
```

```
Accepted connection from 10.0.0.1, port 54540
```

| [ ID] | Interval        | Transfer    | Bitrate        | Jitter   | Lost/Total Datagrams |
|-------|-----------------|-------------|----------------|----------|----------------------|
| [ 7]  | 0.00-1.00 sec   | 1008 KBytes | 8.25 Mbits/sec | 1.989 ms | 0/713 (0%)           |
| [ 7]  | 1.00-2.00 sec   | 1.12 MBytes | 9.42 Mbits/sec | 1.270 ms | 0/813 (0%)           |
| [ 7]  | 2.00-3.00 sec   | 1.12 MBytes | 9.45 Mbits/sec | 0.836 ms | 0/814 (0%)           |
| [ 7]  | 3.00-4.00 sec   | 335 KBytes  | 3.22 Mbits/sec | 1.511 ms | 0/278 (0%)           |
| [ 7]  | 4.00-5.00 sec   | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 5.00-6.00 sec   | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 6.00-7.00 sec   | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 7.00-8.00 sec   | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 8.00-9.00 sec   | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 9.00-10.00 sec  | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 10.00-11.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 11.00-12.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 12.00-13.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 13.00-14.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 14.00-15.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 15.00-16.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 16.00-17.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 17.00-18.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 18.00-19.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 19.00-20.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 20.00-21.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 21.00-22.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 22.00-23.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.511 ms | 0/0 (0%)             |
| [ 7]  | 23.00-24.00 sec | 537 KBytes  | 4.89 Mbits/sec | 0.873 ms | 16501/16923 (98%)    |
| [ 7]  | 24.00-25.00 sec | 1.07 MBytes | 9.01 Mbits/sec | 1.141 ms | 90/866 (10%)         |
| [ 7]  | 25.00-26.00 sec | 451 KBytes  | 3.69 Mbits/sec | 1.981 ms | 16/335 (4.8%)        |
| [ 7]  | 26.00-27.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 27.00-28.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 28.00-29.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 29.00-30.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 30.00-34.25 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 34.25-35.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 35.00-36.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 36.00-37.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 37.00-54.51 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 54.51-55.00 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 55.00-57.08 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |
| [ 7]  | 57.08-81.24 sec | 0.00 Bytes  | 0.00 bits/sec  | 1.981 ms | 0/0 (0%)             |

Il nodo  $H_4$  tenta di saturare la banda per simulare un download intensivo. Pur superando agilmente i 4.0 Mbps di volume, l'utilizzo del protocollo TCP garantisce l'invio di frame di dimensioni massime (MTU), mantenendo il Packet Rate (PPS) al di sotto della soglia critica di 650 PPS. Il controller riconosce questo comportamento come lecito, permettendogli di concludere il test senza alcuna interruzione.



```

root@RETI:/home/giulia/sdn_project# iperf3 -c 10.0.0.3 -t 60 -i 1
Connecting to host 10.0.0.3, port 5201
[ 7] local 10.0.0.4 port 46440 connected to 10.0.0.3 port 5201
[ ID] Interval      Transfer    Bitrate    Retr    Cwnd
[ 7] 0.00-1.01    sec 1.72 MBytes 14.4 Mbits/sec 0    130 KBytes
[ 7] 1.01-2.00    sec 1.37 MBytes 11.6 Mbits/sec 4    180 KBytes
[ 7] 2.00-3.00    sec 1.18 MBytes 9.90 Mbits/sec 0    184 KBytes
[ 7] 3.00-4.00    sec 1.12 MBytes 9.38 Mbits/sec 0    189 KBytes
[ 7] 4.00-5.00    sec 1.18 MBytes 9.90 Mbits/sec 0    194 KBytes
[ 7] 5.00-6.00    sec 891 KBytes 7.30 Mbits/sec 0    214 KBytes
[ 7] 6.00-7.00    sec 1.37 MBytes 11.5 Mbits/sec 0    247 KBytes
[ 7] 7.00-8.00    sec 1.68 MBytes 14.1 Mbits/sec 0    294 KBytes
[ 7] 8.00-9.00    sec 1.43 MBytes 12.0 Mbits/sec 0    359 KBytes
[ 7] 9.00-10.00   sec 827 KBytes 6.77 Mbits/sec 0    445 KBytes
[ 7] 10.00-11.00  sec 1.86 MBytes 15.6 Mbits/sec 0    551 KBytes
[ 7] 11.00-12.00  sec 1.18 MBytes 9.89 Mbits/sec 0    684 KBytes
[ 7] 12.00-13.00  sec 2.50 MBytes 21.0 Mbits/sec 0    843 KBytes
[ 7] 13.00-14.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.00 MBytes
[ 7] 14.00-15.00  sec 2.50 MBytes 21.0 Mbits/sec 0    1.21 MBytes
[ 7] 15.00-16.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.44 MBytes
[ 7] 16.00-17.00  sec 2.50 MBytes 21.0 Mbits/sec 0    1.72 MBytes
[ 7] 17.00-18.00  sec 2.50 MBytes 21.0 Mbits/sec 0    1.99 MBytes
[ 7] 18.00-19.00  sec 2.50 MBytes 21.0 Mbits/sec 0    2.35 MBytes
[ 7] 19.00-20.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.71 MBytes
[ 7] 20.00-21.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.93 MBytes
[ 7] 21.00-22.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.05 MBytes
[ 7] 22.00-23.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.92 MBytes
[ 7] 23.00-24.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.01 MBytes
[ 7] 24.00-25.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.12 MBytes
[ 7] 25.00-26.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.23 MBytes
[ 7] 26.00-27.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.34 MBytes
[ 7] 27.00-28.00  sec 0.00 Bytes 0.00 bits/sec 0    2.43 MBytes
[ 7] 28.00-29.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.50 MBytes
[ 7] 29.00-30.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.55 MBytes
[ 7] 30.00-31.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.60 MBytes
[ 7] 31.00-32.00  sec 0.00 Bytes 0.00 bits/sec 1    2.62 MBytes
[ 7] 32.00-33.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.62 MBytes
[ 7] 33.00-34.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.62 MBytes
[ 7] 34.00-35.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.62 MBytes
[ 7] 35.00-36.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.63 MBytes
[ 7] 36.00-37.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.65 MBytes
[ 7] 37.00-38.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.38 MBytes
[ 7] 38.00-39.00  sec 0.00 Bytes 0.00 bits/sec 0    2.06 MBytes
[ 7] 39.00-40.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.87 MBytes
[ 7] 40.00-41.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.94 MBytes
[ 7] 41.00-42.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.06 MBytes
[ 7] 42.00-43.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.17 MBytes
[ 7] 43.00-44.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.27 MBytes
[ 7] 44.00-45.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.36 MBytes
[ 7] 45.00-46.00  sec 0.00 Bytes 0.00 bits/sec 0    2.44 MBytes
[ 7] 46.00-47.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.50 MBytes
[ 7] 47.00-48.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.55 MBytes
[ 7] 48.00-49.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.58 MBytes
[ 7] 49.00-50.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.61 MBytes
[ 7] 50.00-51.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.62 MBytes
[ 7] 51.00-52.00  sec 0.00 Bytes 0.00 bits/sec 0    2.63 MBytes

```

```

Server listening on 5201
Accepted connection from 10.0.0.4, port 46424
[ 7] local 10.0.0.3 port 5201 connected to 10.0.0.4 port 46440
[ ID] Interval      Transfer    Bitrate
[ 7] 0.00-1.21    sec 1.25 MBytes 8.70 Mbits/sec
[ 7] 1.21-2.20    sec 1.12 MBytes 9.47 Mbits/sec
[ 7] 2.20-3.21    sec 1.12 MBytes 9.40 Mbits/sec
[ 7] 3.21-4.21    sec 1.12 MBytes 9.43 Mbits/sec
[ 7] 4.21-5.23    sec 1.12 MBytes 9.24 Mbits/sec
[ 7] 5.23-6.12    sec 1.00 MBytes 9.40 Mbits/sec
[ 7] 6.12-7.12    sec 1.12 MBytes 9.48 Mbits/sec
[ 7] 7.12-8.22    sec 1.25 MBytes 9.51 Mbits/sec
[ 7] 8.22-9.12    sec 1.00 MBytes 9.31 Mbits/sec
[ 7] 9.12-10.15   sec 1.12 MBytes 9.20 Mbits/sec
[ 7] 10.15-11.16  sec 1.12 MBytes 9.29 Mbits/sec
[ 7] 11.16-12.18  sec 1.12 MBytes 9.28 Mbits/sec
[ 7] 12.18-13.18  sec 1.12 MBytes 9.41 Mbits/sec
[ 7] 13.18-14.19  sec 1.12 MBytes 9.37 Mbits/sec
[ 7] 14.19-15.20  sec 1.12 MBytes 9.33 Mbits/sec
[ 7] 15.20-16.21  sec 1.12 MBytes 9.35 Mbits/sec
[ 7] 16.21-17.12  sec 1.00 MBytes 9.17 Mbits/sec
[ 7] 17.12-18.13  sec 1.12 MBytes 9.39 Mbits/sec
[ 7] 18.13-19.16  sec 1.12 MBytes 9.12 Mbits/sec
[ 7] 19.16-20.19  sec 1.12 MBytes 9.20 Mbits/sec
[ 7] 20.19-21.22  sec 1.12 MBytes 9.16 Mbits/sec
[ 7] 21.22-22.21  sec 1.12 MBytes 9.49 Mbits/sec
[ 7] 22.21-23.22  sec 1.12 MBytes 9.41 Mbits/sec
[ 7] 23.22-24.22  sec 1.12 MBytes 9.37 Mbits/sec
[ 7] 24.22-25.13  sec 1.00 MBytes 9.31 Mbits/sec
[ 7] 25.13-26.21  sec 1.12 MBytes 8.72 Mbits/sec
[ 7] 26.21-27.12  sec 1.00 MBytes 9.18 Mbits/sec
[ 7] 27.12-28.14  sec 1.12 MBytes 9.30 Mbits/sec
[ 7] 28.14-29.36  sec 1.12 MBytes 7.72 Mbits/sec
[ 7] 29.36-30.23  sec 896 KBytes 8.45 Mbits/sec
[ 7] 30.23-31.36  sec 1.00 MBytes 7.40 Mbits/sec
[ 7] 31.36-32.19  sec 896 KBytes 8.89 Mbits/sec
[ 7] 32.19-33.15  sec 896 KBytes 7.64 Mbits/sec
[ 7] 33.15-34.22  sec 1.12 MBytes 8.83 Mbits/sec
[ 7] 34.22-35.12  sec 1.00 MBytes 9.28 Mbits/sec
[ 7] 35.12-36.15  sec 1.12 MBytes 9.21 Mbits/sec
[ 7] 36.15-37.18  sec 1.12 MBytes 9.10 Mbits/sec
[ 7] 37.18-38.15  sec 1.00 MBytes 8.64 Mbits/sec
[ 7] 38.15-39.18  sec 1.12 MBytes 9.22 Mbits/sec
[ 7] 39.18-40.19  sec 1.12 MBytes 9.27 Mbits/sec
[ 7] 40.19-41.12  sec 1.00 MBytes 9.04 Mbits/sec
[ 7] 41.12-42.16  sec 1.12 MBytes 9.14 Mbits/sec
[ 7] 42.16-43.18  sec 1.12 MBytes 9.18 Mbits/sec
[ 7] 43.18-44.15  sec 1.00 MBytes 8.67 Mbits/sec
[ 7] 44.15-45.23  sec 1.12 MBytes 8.76 Mbits/sec
[ 7] 45.23-46.18  sec 1.00 MBytes 8.78 Mbits/sec
[ 7] 46.18-47.17  sec 1.00 MBytes 8.54 Mbits/sec
[ 7] 47.17-48.24  sec 1.12 MBytes 8.82 Mbits/sec
[ 7] 48.24-49.17  sec 1.00 MBytes 8.98 Mbits/sec
[ 7] 49.17-50.20  sec 1.12 MBytes 9.15 Mbits/sec

```

```

[ 7] 52.00-53.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.64 MBytes
[ 7] 53.00-54.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.62 MBytes
[ 7] 54.00-55.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.32 MBytes
[ 7] 55.00-56.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.01 MBytes
[ 7] 56.00-57.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.86 MBytes
[ 7] 57.00-58.00  sec 1.25 MBytes 10.5 Mbits/sec 0    1.91 MBytes
[ 7] 58.00-59.00  sec 0.00 Bytes 0.00 bits/sec 0    1.97 MBytes
[ 7] 59.00-60.00  sec 1.25 MBytes 10.5 Mbits/sec 0    2.04 MBytes
-----
[ ID] Interval      Transfer    Bitrate    Retr    sender receiver
[ 7] 0.00-60.00    sec 74.5 MBytes 10.4 Mbits/sec 5
[ 7] 0.00-62.45    sec 66.5 MBytes 8.93 Mbits/sec
iperf Done.
root@RETI:/home/giulia/sdn_project#

```

```

[ 7] 50.20-51.13  sec 1.00 MBytes 8.99 Mbits/sec
[ 7] 51.13-52.17  sec 1.12 MBytes 9.12 Mbits/sec
[ 7] 52.17-53.14  sec 1.00 MBytes 8.65 Mbits/sec
[ 7] 53.14-54.19  sec 1.12 MBytes 8.97 Mbits/sec
[ 7] 54.19-55.17  sec 1.00 MBytes 8.53 Mbits/sec
[ 7] 55.17-56.13  sec 1.00 MBytes 8.77 Mbits/sec
[ 7] 56.13-57.17  sec 1.00 MBytes 8.08 Mbits/sec
[ 7] 57.17-58.26  sec 1.12 MBytes 8.69 Mbits/sec
[ 7] 58.26-59.11  sec 896 KBytes 8.55 Mbits/sec
[ 7] 59.11-60.14  sec 1.12 MBytes 9.16 Mbits/sec
[ 7] 60.14-61.26  sec 768 KBytes 5.64 Mbits/sec
[ 7] 61.26-62.19  sec 1.00 MBytes 9.01 Mbits/sec
[ 7] 62.19-62.45  sec 256 KBytes 8.00 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 7] 0.00-62.45    sec 66.5 MBytes 8.93 Mbits/sec

```

## 1.3 Conclusioni

Il seguente progetto ha l'obiettivo di dimostrare come l'adozione di un'architettura SDN permetta di superare i limiti strutturali delle reti tradizionali, risolvendo in particolare il problema dell'over-blocking causato dal blocco dell'intera porta di uno switch in caso di attacco volumetrico. Il controller, misurando il volume e la frequenza del traffico ha permesso al sistema di distinguere autonomamente un flusso lecito ad alto volume in TCP da un attacco flood UDP trasmesso 'alla cieca' e ha installato dinamicamente una regola di DROP mirata esclusivamente all'IP dell'attaccante. Infine, il sistema garantisce un'elevata flessibilità applicando alla regola di scarto un hard timeout di 20 secondi: allo scadere di questo periodo di mitigazione, l'host viene sbloccato automaticamente, permettendo il ripristino delle normali attività di rete senza alcun intervento manuale.